SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code: DSA03	Course Title: Natural Language Processing
CO Assessed: CO1 – Imitation (L1)	Assessment: Assessment Tool 1 – Application Based Questions
Weightage: 40% of CIA	Duration: 30 minutes

CO1 Statement: Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)

Student Name: G V Durga Charan	Reg. No.: 192324100
Section / Batch: iii	Date:30-10-2026

Code of Conduct

I G V Durga Charan (192324100) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G V Durga Charan

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.
- Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Answer 1 – Approach

Each field is captured with its own compiled regular expression. A function extract_profile() applies all patterns to one resume and returns a dictionary; is_eligible() checks the shortlisting rule; print_summary() produces the structured profile. Patterns used:

Field	Regex	Idea
Name	^\s*Name\s*[:\-]\s*([A-Za-z][A-Za-z .]+)$	Line beginning with 'Name:'; group 1 is the name
Email	\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b	local part @ domain . TLD
Mobile	(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{9})(?!\d)	10 digits starting 6-9, optional +91 / 0
Skills	\b(Python|Java|SQL|Machine\s+Learning|NLP)\b	Alternation, case-insensitive
Experience	(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b	Number followed by year(s)/yrs

Program

```
import re

# ---------- Compiled regular expression patterns ----------
NAME_PATTERN  = re.compile(r"^\s*Name\s*[:\-]\s*([A-Za-z][A-Za-z .]+)$", re.MULTILINE |
re.IGNORECASE)
EMAIL_PATTERN = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
MOBILE_PATTERN = re.compile(r"(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{9})(?!\d)")
SKILL_PATTERN = re.compile(r"\b(Python|Java|SQL|Machine\s+Learning|NLP)\b", re.IGNORECASE)
EXPERIENCE_PATTERN = re.compile(r"(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b", re.IGNORECASE)

MIN_EXPERIENCE = 2
REQUIRED_SKILL = "python"


def extract_profile(resume_text):
    """Extract all required fields from one resume and return a dictionary."""
    name = NAME_PATTERN.search(resume_text)
    emails = EMAIL_PATTERN.findall(resume_text)
```

```
    mobiles = MOBILE_PATTERN.findall(resume_text)
    skills = sorted({re.sub(r"\s+", " ", s).title() if s.lower() != "nlp" and s.lower() !=
"sql"
                     else s.upper() for s in SKILL_PATTERN.findall(resume_text)})
    exp = [float(x) for x in EXPERIENCE_PATTERN.findall(resume_text)]
    return {
        "name": name.group(1).strip() if name else "Not found",
        "emails": emails or ["Not found"],
        "mobiles": mobiles or ["Not found"],
        "skills": skills or ["None"],
        "experience": max(exp) if exp else 0.0,
    }


def is_eligible(profile):
    """Eligibility: at least 2 years of experience AND Python skill."""
    has_python = any(s.lower() == REQUIRED_SKILL for s in profile["skills"])
    return profile["experience"] >= MIN_EXPERIENCE and has_python


def print_summary(profile):
    print(f"Name       : {profile['name']}")
    print(f"Email      : {', '.join(profile['emails'])}")
    print(f"Mobile     : {', '.join(profile['mobiles'])}")
    print(f"Skills     : {', '.join(profile['skills'])}")
    print(f"Experience : {profile['experience']:g} year(s)")
    print(f"Eligible   : {'YES' if is_eligible(profile) else 'NO'}")


resumes = [
"""Name: Arun Kumar
Email: arun.kumar@gmail.com
Mobile: +91 9876543210
Skills: Python, SQL, Machine Learning
Experience: 3 years in data analytics""",
"""Name: Priya Sharma
Email: priya_s@yahoo.com
Phone: 8123456789
Skills: Java, SQL
Experience: 5 years as backend developer""",
"""Name: Rahul Menon
Email: rahul.menon@outlook.com
Mobile: 9012345678
Skills: Python, NLP
Experience: 1 year as intern""",
"""Name: Divya R
Email: divya.r@mail.com
Mobile: 7001234567
Skills: Python, Java, NLP, Machine Learning
Experience: 4.5 years in AI research""",
]

if __name__ == "__main__":
    shortlisted = []
    for number, text in enumerate(resumes, start=1):
        print(f"\n--- Candidate {number} ---")
        try:
            profile = extract_profile(text)
            print_summary(profile)
            if is_eligible(profile):
                shortlisted.append(profile["name"])
        except Exception as err:          # error handling
            print("Could not process resume:", err)
```

```
    print("\n===== SHORTLISTED CANDIDATES (>= 2 yrs & Python) =====")
    for n in shortlisted:
        print("*", n)
```

Output

```
--- Candidate 1 ---
Name       : Arun Kumar
Email      : arun.kumar@gmail.com
Mobile     : 9876543210
Skills     : Machine Learning, Python, SQL
Experience : 3 year(s)
Eligible   : YES

--- Candidate 2 ---
Name       : Priya Sharma
Email      : priya_s@yahoo.com
Mobile     : 8123456789
Skills     : Java, SQL
Experience : 5 year(s)
Eligible   : NO

--- Candidate 3 ---
Name       : Rahul Menon
Email      : rahul.menon@outlook.com
Mobile     : 9012345678
Skills     : NLP, Python
Experience : 1 year(s)
Eligible   : NO

--- Candidate 4 ---
Name       : Divya R
Email      : divya.r@mail.com
Mobile     : 7001234567
Skills     : Java, Machine Learning, NLP, Python
Experience : 4.5 year(s)
Eligible   : YES

===== SHORTLISTED CANDIDATES (>= 2 yrs & Python) =====
* Arun Kumar
* Divya R
```

Justification

- Arun Kumar (3 yrs, Python) and Divya R (4.5 yrs, Python) meet both criteria, so they are shortlisted.
- Priya Sharma has 5 years of experience but no Python skill, so she is rejected.
- Rahul Menon knows Python but has only 1 year of experience (< 2), so he is rejected.
- The +91 prefix is removed because only the 10-digit group is captured; 4.5 is read as a decimal by the experience pattern.

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

- Search products using an exact keyword.
- Perform prefix-based searches.

- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

Answer 2 – Approach

The keyword is escaped with re.escape() so special characters are treated literally, and a pattern is built for the chosen mode. re.IGNORECASE makes every search case-insensitive.

Mode	Pattern	Meaning
Exact	\bkeyword\b	Keyword as a complete word
Prefix	\bkeyword\w*	A word that starts with the keyword
Suffix	\w*keyword\b	A word that ends with the keyword
Partial	keyword	Keyword anywhere in the name

Program

```
import re

products = ["Laptop", "Gaming Laptop", "Laptop Bag", "Smartphone", "Smart Watch",
            "Wireless Mouse", "Mouse Pad", "Bluetooth Speaker", "Headphones",
            "Wireless Headphones", "Keyboard", "Mechanical Keyboard", "Phone Case"]


def build_pattern(keyword, mode):
    """Return a compiled case-insensitive pattern for the chosen search mode."""
    word = re.escape(keyword)
    patterns = {
        "exact":   rf"\b{word}\b",           # whole word anywhere in name
        "prefix":  rf"\b{word}\w*",          # word starting with keyword
        "suffix":  rf"\w*{word}\b",          # word ending with keyword
        "partial": rf"{word}",               # keyword anywhere
    }
    if mode not in patterns:
        raise ValueError("Unknown search mode: " + mode)
    return re.compile(patterns[mode], re.IGNORECASE)   # case-insensitive


def search(keyword, mode):
    if not keyword.strip():
        raise ValueError("Keyword cannot be empty")
    pattern = build_pattern(keyword, mode)
    return [p for p in products if pattern.search(p)]


tests = [("laptop", "exact"), ("SMART", "prefix"), ("phones", "suffix"),
         ("board", "partial"), ("WIRELESS", "exact"), ("tablet", "exact")]

report = []
for keyword, mode in tests:
    results = search(keyword, mode)
    print(f"\nSearch: '{keyword}'  Mode: {mode}")
    print("Matches:", results if results else "No products found")
    report.append((keyword, mode, len(results)))

print("\n========== SEARCH REPORT ==========")
print(f"{'Keyword':<10}{'Mode':<10}{'Total Matches'}")
```

```
for k, m, c in report:
    print(f"{k:<10}{m:<10}{c}")
```

Output

```
Search: 'laptop'  Mode: exact
Matches: ['Laptop', 'Gaming Laptop', 'Laptop Bag']

Search: 'SMART'  Mode: prefix
Matches: ['Smartphone', 'Smart Watch']

Search: 'phones'  Mode: suffix
Matches: ['Headphones', 'Wireless Headphones']

Search: 'board'  Mode: partial
Matches: ['Keyboard', 'Mechanical Keyboard']

Search: 'WIRELESS'  Mode: exact
Matches: ['Wireless Mouse', 'Wireless Headphones']

Search: 'tablet'  Mode: exact
Matches: No products found

========== SEARCH REPORT ==========
Keyword   Mode      Total Matches
laptop    exact     3
SMART     prefix    2
phones    suffix    2
board     partial   2
WIRELESS  exact     2
tablet    exact     0
```

Justification

- 'laptop' (exact) matches Laptop, Gaming Laptop and Laptop Bag – 3 products; the case difference is ignored.
- 'SMART' (prefix) matches Smartphone and Smart Watch because both words begin with 'smart'.
- 'phones' (suffix) matches Headphones and Wireless Headphones, which end with 'phones'; Smartphone is excluded as it ends with 'phone'.
- 'board' (partial) finds it inside Keyboard and Mechanical Keyboard.
- 'tablet' is not in the catalogue, so the count is 0 and a 'No products found' message is shown.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

Answer 3 – Approach

The problem statement does not fix the formats, so the following reasonable rules are assumed. Each field has a pattern checked with fullmatch(), so the whole value must conform.

Field	Regex	Rule
Register Number	^19\d{7}$	9 digits starting with 19
Email	^[A-Za-z0-9._%+-]+@(?:saveetha\.com|simats\.edu\.in)$	Institutional domain only
Course Code	^[A-Z]{3}\d{2}$	3 capital letters + 2 digits (DSA03)
Semester	^(?:Semester\s|Sem\s)?[1-8]$	1 to 8, optional 'Sem' prefix
Mobile	^(?:\+91[\s-]?)?[6-9]\d{9}$	10 digits starting 6-9, optional +91

Program

```
import re

RULES = {
    "Register Number": (re.compile(r"^19\d{7}$"),
                        "9 digits starting with 19 (e.g. 192424001)"),
    "Email":           (re.compile(r"^[A-Za-z0-9._%+-]+@(?:saveetha\.com|simats\.edu\.in)$"),
                        "must end with @saveetha.com or @simats.edu.in"),
    "Course Code":     (re.compile(r"^[A-Z]{3}\d{2}$"),
                        "3 capital letters + 2 digits (e.g. DSA03)"),
    "Semester":        (re.compile(r"^(?:Semester\s|Sem\s)?[1-8]$", re.IGNORECASE),
                        "number 1-8, optionally prefixed by 'Sem'/'Semester'"),
    "Mobile":          (re.compile(r"^(?:\+91[\s-]?)?[6-9]\d{9}$"),
                        "10 digits starting with 6-9, optional +91"),
}


def validate_student(student):
    """Validate every field, print messages and return overall status."""
    all_valid = True
    for field, (pattern, hint) in RULES.items():
        value = student.get(field, "").strip()
        if pattern.fullmatch(value):
            print(f"  [VALID]   {field}: {value}")
        else:
            print(f"  [INVALID] {field}: '{value}' -> {hint}")
            all_valid = False
    return all_valid


students = [
    {"Register Number": "192424001", "Email": "john.doe@saveetha.com",
     "Course Code": "DSA03", "Semester": "Semester 5", "Mobile": "+91 9876543210"},
    {"Register Number": "19242400", "Email": "mary@gmail.com",
     "Course Code": "dsa3", "Semester": "9", "Mobile": "5123456789"},
]

for i, s in enumerate(students, 1):
    print(f"\nStudent {i} Validation")
    try:
        ok = validate_student(s)
    except Exception as err:
        print("Error:", err); ok = False
    print("FINAL REGISTRATION STATUS:", "SUCCESSFUL" if ok else "FAILED")
```

Output

```
Student 1 Validation
  [VALID]   Register Number: 192424001
  [VALID]   Email: john.doe@saveetha.com
  [VALID]   Course Code: DSA03
  [VALID]   Semester: Semester 5
  [VALID]   Mobile: +91 9876543210
FINAL REGISTRATION STATUS: SUCCESSFUL

Student 2 Validation
  [INVALID] Register Number: '19242400' -> 9 digits starting with 19 (e.g. 192424001)
  [INVALID] Email: 'mary@gmail.com' -> must end with @saveetha.com or @simats.edu.in
  [INVALID] Course Code: 'dsa3' -> 3 capital letters + 2 digits (e.g. DSA03)
  [INVALID] Semester: '9' -> number 1-8, optionally prefixed by 'Sem'/'Semester'
  [INVALID] Mobile: '5123456789' -> 10 digits starting with 6-9, optional +91
FINAL REGISTRATION STATUS: FAILED
```

Justification

- Student 1 satisfies all five patterns, so every field is VALID and registration is SUCCESSFUL.
- Student 2 fails every pattern: 8-digit register number, Gmail address (not institutional), lowercase course code 'dsa3', semester 9 (outside 1-8) and a mobile number starting with 5. Registration therefore FAILS.
- Registration is successful only when all fields are valid, tracked by the all_valid flag.

Rubrics for Calculation

Criteria	Weightage	Level 4 ? Exemplary	Level 3 ? Proficient	Level 2 ? Developing	Level 1 ? Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity
1					
2					
3					

Sub. Total: _____ Total %: _____

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____